

SUPSI

Magic Eraser

Studente/i

Abate Luca

Vavassori Enrico

Relatore

Gremlich Giuliano

Correlatore

Quattrini Andrea

Committente

**Video processing and computer
vision - Scientific Sector (ISIN)**

Corso di laurea

Ingegneria Informatica TP

Modulo

Progetto di Semestre

Anno

2026

Data

2 maggio 2026

Indice

Introduzione	iii
1 Motivazione e Contesto	1
1.1 Rilevanza del problema	1
1.2 Contesto tecnologico	1
1.3 Obiettivo del progetto	1
2 Definizione del Problema	3
2.1 Aspetti del problema	3
2.2 Conseguenze operative	4
3 Stato dell'Arte	5
3.1 Segmentazione e maschere	5
3.1.1 Convex hull delle maschere	7
3.2 Image Inpainting	8
3.3 Modelli generativi	10
3.4 Confronto con strumenti esistenti	12
4 Approccio alla Soluzione	13
4.1 Descrizione generale	13
4.2 Architettura del sistema	13
4.2.1 Frontend	13
4.2.2 Backend	13
4.2.3 Database	14
4.2.4 AI provider	14
4.3 Pipeline di elaborazione	14
4.4 Scelte tecnologiche	14
5 Implementazione	15
5.1 Struttura del sistema	15
5.2 Frontend	15
5.3 Backend e API	15

5.4	Gestione immagini e progetti	16
5.5	Pipeline AI	16
5.5.1	Segmentazione	16
5.5.2	Rimozione	16
5.5.3	Generazione	16
5.6	Organizzazione del codice	16
6	Risultati e Validazione	17
6.1	Funzionalità implementate	17
6.2	Test	17
6.3	Validazione	17
6.4	Risultati	18
7	Conclusioni	19
7.1	Valutazione del lavoro	19
7.2	Limiti della soluzione	19
7.3	Contributo	19
7.4	Sviluppi futuri	20
A	Manuale Utente	22
A.1	Login	22
A.2	Creare un progetto	22
A.3	Caricare un'immagine	22
A.4	Usare il laboratorio	22
A.5	Salvare una pipeline	22
B	API principali	23
B.1	Elenco endpoint	23
C	Configurazione e Avvio	24
C.1	Docker Compose	24
C.2	Variabili ambiente	24
C.3	Avvio frontend e backend	24

Introduzione

Abstract

Italiano

Magic Eraser è uno strumento per la modifica di immagini tramite modelli di intelligenza artificiale. Il progetto nasce come tool utilizzato per ricerca, sperimentazione, analisi e valutazione di modelli per segmentazione, rimozione e generazione; nello specifico per quanto riguarda la rimozione automatica di oggetti e alla modifica di immagini tramite maschere. La soluzione proposta combina gestione dei progetti, caricamento delle immagini in essi, segmentazione da prompt, rimozione tramite maschera e generazione di nuove aree coerenti con il contesto visivo.

Il sistema è organizzato come applicazione web con frontend React (TypeScript), backend FastAPI, persistenza dati su Supabase Postgres e integrazione con provider AI esterni (tra cui Fal.ai).

L'elaborazione delle immagini è gestita tramite pipeline persistite, in modo da rendere tracciabili i passaggi applicati e permettere di salvare, modificare e riutilizzare i risultati.

Il prodotto risultante permette la creazione di progetti, caricamento di immagini, applicazione di processi AI (segmentazione, rimozione, generazione) tramite provider esterni su esse, e di mantenere una cronologia ordinata delle elaborazioni.

Magic Eraser può essere anche utilizzato come strumento di modifica assistita di immagini per semplificare e rendere più accessibile operazioni avanzate. Questo tenendo in considerazione che i risultati prodotti restano dipendenti dalla qualità dei modelli esterni, dalla precisione delle maschere e dai tempi di risposta dei servizi AI. Magic Eraser rappresenta una base estendibile per la sperimentazione di workflow di modifica immagini con un approccio generativo e per il confronto di modelli differenti in situazioni reali.

English

Magic Eraser is a tool for editing images using artificial intelligence models. The project began as a tool for research, experimentation, analysis and evaluation of models for segmentation, removal and generation; specifically with regard to the automatic removal of ob-

jects and the modification of images using masks. The proposed solution combines project management, the uploading of images to projects, segmentation from prompts, removal via masks. The proposed solution combines project management, the uploading of images to these projects, segmentation based on prompts, removal using masks, and the generation of new areas that are consistent with the visual context.

The system is structured as a web application with a React (TypeScript) frontend, a FastAPI backend, data storage on Supabase Postgres, and integration with external AI providers (including Fal.ai).

Image processing is managed via persistent pipelines, so that the steps applied can be traced and the results can be saved, modified and reused.

The resulting product enables users to create projects, upload images, apply AI processes (segmentation, removal, generation) to them via external providers, and maintain a clear history of the processing steps.

Magic Eraser can also be used as an assisted image editing tool to simplify advanced operations and make them more accessible. This is bearing in mind that the results produced depend on the quality of the external models, the precision of the masks and the response times of the AI services. Magic Eraser serves as a flexible platform for experimenting with image editing workflows using a generative approach and for comparing different models in real-world scenarios.

Struttura del documento

Il documento è strutturato come segue: vengono prima chiariti contesto e problema, successivamente vengono descritte le tecnologie, architetture e approcci adottati, per poi passare all'implementazione e ai risultati. In conclusione le appendici che contengono informazioni, guide e istruzioni per utilizzare Magic Eraser.

Capitolo 1

Motivazione e Contesto

Nel corso dell'ultimo decennio, la diffusione di strumenti per la modifica di immagini basati su Intelligenza Artificiale ha reso accessibili operazioni che, nella maggior parte dei casi, richiedevano competenze grafiche avanzate. Magic Eraser si inserisce in questo contesto come uno strumento che, non solo automatizza e semplifica le operazioni precedentemente citate, ma permette anche il confronto tra modelli e processi di diversa natura.

1.1 Rilevanza del problema

La modifica tradizionale di immagini richiede conoscenza di strumenti professionali e spesso complicati, selezioni manuali e tempo per rifinire il risultato. In molte situazioni reali, come la preparazione di immagini per presentazioni, contenuti per social media, materiale didattico o prototipi visivi, si ha bisogno di un risultato rapido e il meno macchinoso possibile. Un sistema automatico di rimozione oggetti può ridurre la complessità, lasciando all'utente il controllo sull'idea e intenzione, delegando ai modelli AI le operazioni più tecniche e "noiose".

1.2 Contesto tecnologico

Magic Eraser si colloca tra AI generativa e computer vision. Segmentazione e maschere rientrano nel contesto di Pattern Recognition e Computer Vision e hanno lo scopo di individuare e isolare le aree rilevanti di un'immagine. Mentre i modelli generativi consentono di ricostruire contenuti coerenti nelle zone interessate, nel caso di Magic Eraser, coincidenti con le maschere.

1.3 Obiettivo del progetto

L'obiettivo, da un punto di vista macroscopico, è sviluppare un sistema che permetta di caricare immagini personali, selezionare i processi AI che si desidera usare, generare maschere, rimuovere contenuti e salvare i risultati all'interno di progetti organizzati. L'utilizzo

deve essere sufficientemente semplice per un utente non tecnico, ma anche predisposto per analisi e sperimentazione sui modelli impiegati.

Capitolo 2

Definizione del Problema

Il problema affrontato dal progetto riguarda la gestione completa di un processo di modifica immagini basato su modelli AI. In particolare, non si tratta soltanto di applicare un effetto automatico a un'immagine, come potrebbe essere fatto tramite smartphone in pochi istanti, ma di coordinare più passaggi collegati tra loro: selezione dell'area da modificare, generazione di una maschera da tale area, rimozione e ricostruzione, generazione di una nuova entità nella stessa area, recupero del risultato per ogni processo da modelli esterni e salvataggio dell'intera pipeline.

Questi passaggi introducono diversi aspetti critici. L'immagine di partenza deve essere in formati standard per poter essere processata dai modelli, le maschere devono essere associate correttamente all'immagine originale, questo significa anche che devono essere coerenti con le caratteristiche di essa (dimensione, formato, qualità), e ogni pipeline deve poter essere ricostruita in seguito. Senza una struttura pensata, il flusso può diventare difficile da controllare, soprattutto quando l'utente lavora su più immagini e/o prova più modelli.

2.1 Aspetti del problema

- **Selezione dell'area da modificare.** Per rimuovere o generare contenuto in una porzione specifica dell'immagine serve una maschera coerente. Una maschera imprecisa può includere parti che si desidera invece conservare, oppure escludere parti che dovrebbero essere modificate, producendo risultati indesiderati anche quando il modello generativo funziona correttamente. Allo stesso tempo, una maschera troppo specifica e dettagliata può portare a risultati ambigui, questo è spiegato dal fatto che i modelli di rimozione e filling vanno a modificare unicamente i pixel presenti nella maschera.
- **Dipendenza da modelli esterni.** I provider AI, e gli stessi modelli, possono richiedere formati di input differenti, tempi di elaborazione variabili, costi elevati e parametri specifici. Il sistema deve quindi tradurre le operazioni richieste dall'utente in chia-

mate tecniche compatibili con il modello scelto, gestendo allo stesso tempo stati di avanzamento e output prodotti.

- **Tracciabilità.** In un contesto di sperimentazione non è sufficiente ottenere un'immagine finale per trarre conclusioni sulla qualità di un processo. Bisogna infatti sapere quale immagine sorgente è stata usata, quale processo è stato applicato, quale modello ha prodotto il risultato e quali dati intermedi sono stati generati. Senza queste informazioni diventa difficile confrontare risultati, ripetere un'elaborazione o capire da dove derivi un certo output.

2.2 Conseguenze operative

Dal punto di vista operativo, il problema richiede un sistema capace di organizzare immagini, versioni e risultati in modo ordinato, strutturato e comprensibile. Ogni progetto deve poter contenere immagini e ogni immagine deve poter essere collegata alle elaborazioni eseguite. Inoltre, l'utente deve poter distinguere gli step della pipeline, visualizzare i risultati intermedi e ritrovare le lavorazioni precedenti.

La difficoltà principale consiste quindi nel trasformare un insieme di operazioni tecniche e potenzialmente poco comprensibili, in un flusso controllabile. Magic Eraser affronta questo problema costruendo una struttura applicativa in cui immagini, maschere, processi AI e output restano collegati tra loro e possono essere consultati anche dopo l'esecuzione e nel corso di sessioni differenti.

Capitolo 3

Stato dell'Arte

Lo stato dell'arte considera le tecniche e gli strumenti che vengono utilizzati nell'ambito dell'editing generativo: image inpainting, segmentazione, maschere e modelli prompt-based. Questi concetti costituiscono la base teorica e applicativa della soluzione sviluppata. Viene inoltre fatta un'analisi comparativa con strumenti simili a Magic Eraser già presenti sul mercato, evidenziando punti di forza e limiti di ciascuno.

3.1 Segmentazione e maschere

La segmentazione consente di individuare regioni dell'immagine corrispondenti a oggetti o concetti indicati dall'utente. Il risultato viene rappresentato tramite maschere, immagini binarie o semi-binarie che separano le zone da modificare dal resto della scena. Nel workflow di Magic Eraser, la maschera è l'elemento di collegamento tra la selezione semantica e l'operazione di rimozione o generativa successiva.

Nel paper dove viene introdotto SAM (*Segment Anything Model*) - Meta AI (2023), il progetto Segment Anything viene descritto come composto da task, modello ed dataset per la segmentazione di immagini [1]. L'idea centrale è la segmentazione guidata da prompt: il task è definito come la capacità di "ottenere una maschera di segmentazione valida dato qualsiasi prompt" [1]. In questa prospettiva, il prompt non si limita a una descrizione testuale, ma può contenere informazioni spaziali o semantiche, come punti, box, maschere o testo libero. Per Magic Eraser questo aspetto viene sfruttato perché l'utente non deve disegnare manualmente la regione da modificare ma deve descrivere tramite testo l'oggetto o l'area su cui desidera apportare modifiche, lasciando al modello il compito di tradurre questa descrizione in una maschera coerente con l'immagine.

Il Segment Anything Model (SAM) è progettato per separare l'analisi dell'immagine dalla generazione della maschera. Il modello usa un *image encoder*, un *prompt encoder* e un *mask decoder*, come mostrato in Figura 3.2; questa struttura permette di calcolare una rappresentazione dell'immagine e poi riutilizzarla con prompt diversi. Gli autori indicano

esplicitamente l'uso di “point, box, and mask prompts” [1], cioè input compatibili con un'interazione visuale rapida. Dal punto di vista applicativo, questa scelta rende la segmentazione adatta a strumenti in cui l'utente deve provare selezioni diverse sulla stessa immagine senza ripetere ogni volta l'intero processo.



Figura 3.1: Esempio di segmentazione tramite prompt. L'immagine è tratta direttamente dal paper *Segment Anything* (SAM) [1].

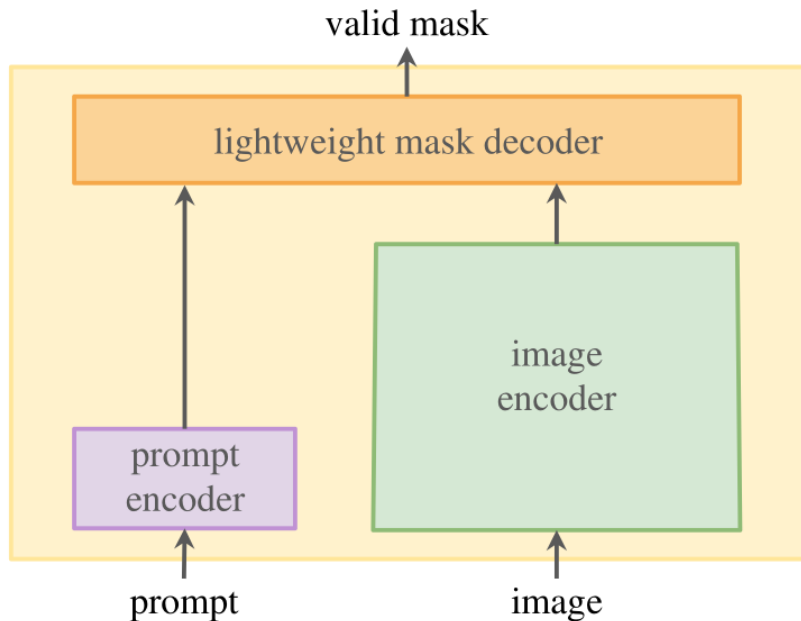


Figura 3.2: Architettura generale del Segment Anything Model: image encoder, prompt encoder e mask decoder. L'immagine è tratta direttamente dal paper *Segment Anything* (SAM) [1].

Un punto importante per l'editing generativo è la gestione dell'ambiguità. Un prompt può riferirsi a oggetti diversi o a parti diverse della stessa scena: ad esempio un punto sopra una maglietta può indicare la maglietta, la persona o una regione più ampia. Per questo motivo SAM può produrre più maschere valide per un singolo prompt, associando anche un punteggio di confidenza. Nel contesto di Magic Eraser, questa caratteristica è utile perché la qualità dell'inpainting dipende direttamente dalla precisione della maschera: una maschera troppo ampia può rimuovere elementi da conservare, mentre una maschera troppo stretta può lasciare residui visivi dell'oggetto.

3.1.1 Convex hull delle maschere

La convex hull di una maschera è la più piccola area convessa che contiene tutti i punti della maschera originale. In pratica, partendo da una selezione anche molto irregolare, la convex hull produce un contorno più ampio e continuo, come mostrato in Figura 3.3. Questo non sostituisce la maschera generata dal modello, ma può essere utile quando si vuole ottenere un'area di intervento più stabile e meno frammentata.

Nel contesto di Magic Eraser, la convex hull viene usata come supporto operativo per trasformare maschere molto dettagliate in regioni più semplici da usare nei processi successivi. Una maschera segmentata con bordi dettagliati e specifici rischia di fornire un'area di modifica troppo precisa per il modello e, specialmente nel caso in cui da segmentazione si passi

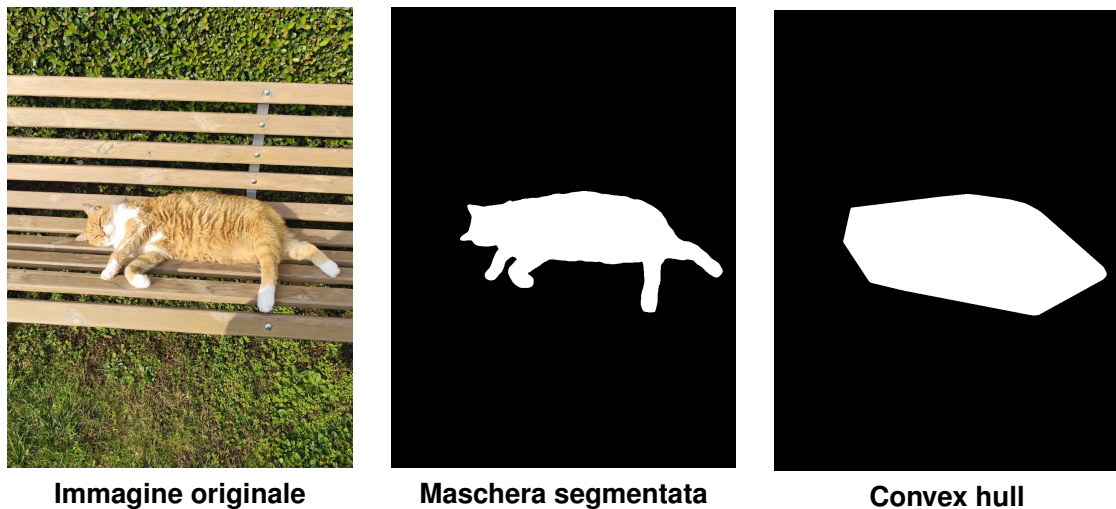


Figura 3.3: Confronto tra immagine originale, maschera segmentata e convex hull della maschera.

direttamente a inpainting filling (senza passare prima dal processo di rimozione oggetto), il risultato è spesso poco soddisfacente. Una convex hull, invece, generalizza la forma della maschera, creando un'area più ampia e continua che può essere più adatta a processi di rimozione o generazione, soprattutto quando si vuole evitare imprecisioni dovuti a bordi troppo irregolari.

3.2 Image Inpainting

L'immagine inpainting è il processo di ricostruzione di aree mancanti, danneggiate o mascherate di un'immagine. Nel paper *Image Inpainting*, viene definito l'inpainting come la tecnica per "modificare un'immagine in una forma non rilevabile" [3]. Spesso l'operazione di inpainting viene associata al restauro artistico e fotografico, spiegando che l'obiettivo è ricostruire parti mancanti o rovinate dell'immagine in modo da renderla nuovamente coerente.

L'interazione richiesta all'utente è limitata alla selezione della regione da correggere e come si desidera riempire. "Dopo che l'utente seleziona le regioni da restaurare, l'algoritmo riempie automaticamente tali regioni usando l'informazione circostante" [3]. In questo senso, l'inpainting è un processo che parte da un'immagine e una maschera, e produce un risultato in cui la regione coperta dalla maschera è stata sostituita con un contenuto coerente con il contesto visivo.



Figura 3.4: Esempio di image inpainting: immagine originale, maschera del tamburello e risultato dopo la rimozione.

Importante è anche distinguere l'inpainting dalla denoising classica. Nel denoising i pixel degradati contengono ancora informazione utile, mentre nell'inpainting la regione da ricostruire può non contenere informazione significativa. Per questo motivo il contenuto va dedotto dalle zone circostanti. L'area coperta dalla maschera non deve essere migliorata, ma sostituita con un contenuto coerente con il contesto.

Tuttavia, uno dei problemi principali, come anche menzionato nel paper, è la “riproduzione di grandi regioni testurizzate” [3]. Questo limite resta rilevante anche nei sistemi moderni: le aree con pattern complessi, sfondi molto ricchi o dettagli ripetitivi possono produrre imprecisioni, soprattutto quando la maschera è grande o poco precisa.



Figura 3.5: Esempio di limite dell'immagine inpainting su una regione testurizzata: la ripetizione delle mattonelle rende più difficile produrre un risultato coerente.

Nei sistemi moderni, l'inpainting non si limita più a modelli basati su diffusione anisotropa o propagazione geometrica, ma utilizza anche modelli generativi capaci di interpretare il

contesto visivo. Tuttavia, il principio operativo rimane lo stesso: data un'immagine e una maschera, il sistema deve produrre una regione plausibile, coerente con il resto della scena e utile all'obiettivo dell'utente.

3.3 Modelli generativi

I modelli generativi image-to-image permettono di produrre o modificare contenuti visivi a partire da istruzioni testuali, immagini di riferimento e maschere. Questo rende il problema diverso dalla generazione di un'immagine da zero, perché il risultato deve rispettare la prospettiva, la luce, la texture e la composizione dell'immagine di partenza.

La famiglia di modelli che ha reso possibile questo tipo di workflow è quella dei modelli a diffusione. In generale, un modello di diffusione impara a ricostruire un'immagine partendo da rumore, procedendo per passi successivi fino a ottenere una scena coerente. Stable Diffusion ha reso questo approccio molto diffuso perché sposta una parte importante del processo nello spazio latente: invece di lavorare direttamente su tutti i pixel dell'immagine, il modello opera su una rappresentazione più compatta, riducendo il costo computazionale e rendendo più accessibili operazioni come image-to-image e inpainting [2]. L'articolo che abbiamo deciso di utilizzare per questa sezione descrive proprio "Stable Diffusion non è solo un modello, ma un ecosistema di varianti, interfacce e tecniche di controllo che hanno reso l'immagine generation utilizzabile anche in contesti applicativi" [5].

Per quanto riguarda Magic Eraser, l'aspetto più importante non è la generazione libera, ma l'editing controllato. Il modello riceve tre informazioni principali: l'immagine originale, la maschera e, quando necessario, un prompt. La maschera limita l'area di modifica, mentre il prompt orienta il tipo di contenuto da generare. Se l'obiettivo è rimuovere un oggetto, il modello deve riempire l'area con uno sfondo coerente e credibile; se l'obiettivo è sostituire o aggiungere contenuto, deve invece generare un elemento nuovo senza "rompere" la coerenza della scena. Questa distinzione è importante perché spiega la presenza di due processi separati presenti: `remove_with_mask` e `generate_from_prompt`.

Tra i modelli usati nell'ambito dell'AI image generation, FLUX (di Black Forest Labs), è considerato, anche da Black Forest Labs stessa, lo stato dell'arte. Sempre nell'articolo citato, vengono descritte le versioni più recenti della famiglia FLUX come modelli orientati alla qualità visiva e alla fedeltà al prompt. [5]. Magic Eraser integra *FLUX.1 [pro] Fill* tramite Fal.ai per il processo `generate_from_prompt`. Questo modello viene usato quando l'utente vuole riempire una regione mascherata seguendo una descrizione testuale. Non genera quindi un'intera immagine indipendente, ma lavora dentro un'area precisa, come detto prima, usando il contesto visivo attorno alla maschera come vincolo.

Accanto a FLUX, l'applicazione, al momento attuale, implementa anche *Bria GenFill* come modello alternativo per la generazione con maschera. La sua presenza è utile perché permette di confrontare risultati diversi sullo stesso input (stessa immagine, stessa maschera, stesso prompt, ma modello differente). Questo è uno degli obiettivi pratici di Magic Eraser: rendere esplicito quali componenti hanno prodotto un certo risultato. In questo modo l'utente può capire se un risultato poco convincente dipende dalla maschera, dal prompt o dal comportamento del modello scelto.

Il processo di rimozione usa invece *Finegrain Eraser*, sempre tramite Fal.ai. Anche se dal punto di vista dell'utente appare come una semplice cancellazione, il modello effettivamente genera contenuto nuovo nella zona indicata con maschera. La differenza rispetto al filling guidato da prompt è che qui l'intenzione principale è far sparire l'oggetto, non sostituirlo con un elemento specifico. Per questo il prompt non è necessario: la maschera indica cosa rimuovere e il modello usa lo sfondo circostante per produrre una continuazione coerente.

La qualità del risultato dipende da più fattori. Anche nell'articolo viene infatti evidenziato che i modelli di generazione immagini sono particolarmente sensibili alla formulazione del prompt, ai parametri di inferenza e alla complessità del contenuto da produrre [5]. Una maschera troppo stretta può lasciare bordi o residui dell'oggetto; una maschera troppo ampia può costringere il modello a inventare una porzione eccessiva dell'immagine. Per questo la pipeline combina segmentazione, (convex hull), rimozione e generazione: ogni passaggio prepara meglio il successivo.

Un limite da considerare è che questi modelli non garantiscono sempre un risultato deterministico e perfetto. Sfondi ripetitivi, superfici con pattern regolari, mani, testi e dettagli geometrici possono ancora produrre imperfezioni. Inoltre, l'uso di provider esterni introduce aspetti pratici come totale dipendenza dalle API disponibili, latenza e costo delle richieste. Magic Eraser non elimina questi limiti, ma li rende osservabili: salvando input, maschere, modello scelto, impostazioni e output, permette di confrontare tentativi diversi e di ragionare sul workflow invece che solo sull'immagine finale.

3.4 Confronto con strumenti esistenti

Strumento	Analisi sintetica
Editor professionali (Photoshop, Krita, Photopea)	Offrono controllo preciso e manuale tramite strumenti avanzati, ma richiedono competenze specifiche e spesso licenze costose.
Strumenti automatici (Canva PRO, CapCut)	Consentono una rimozione rapida degli oggetti, ma nascondono completamente la pipeline e i modelli usati. Spesso le versioni gratuite presentano limitazioni.
Demo di modelli AI (Fal.ai, Replicate)	Permettono di provare singoli modelli individualmente, senza offrire gestione di progetti e immagini al di fuori del singolo test.

Capitolo 4

Approccio alla Soluzione

La soluzione proposta è una piattaforma web composta da interfaccia utente, API backend, database, storage immagini e integrazione con provider AI. L'architettura separa le responsabilità principali, rendendo il sistema più manutenibile e predisposto all'estensione con nuovi modelli o nuovi processi.

4.1 Descrizione generale

L'utente accede all'applicazione, crea o seleziona un progetto, carica immagini e apre il laboratorio di editing. Nel laboratorio può aggiungere processi alla pipeline, scegliere il modello, impostare prompt o parametri aggiuntivi e generare un risultato. Le pipeline vengono salvate con i relativi step, input, output, maschere e stato di elaborazione.

4.2 Architettura del sistema

4.2.1 Frontend

Il frontend è sviluppato con React e Vite. Le pagine principali includono login, signup, home, gallery, laboratory, pipelines e profile. I componenti gestiscono la navigazione, la visualizzazione dei progetti, l'upload delle immagini, la configurazione dei processi e la consultazione delle pipeline salvate.

4.2.2 Backend

Il backend è sviluppato con FastAPI e fornisce endpoint REST per progetti, immagini, profilo utente, processi AI e pipeline di laboratorio. La logica applicativa è organizzata in servizi, repository, schemi di validazione e router API.

4.2.3 Database

La persistenza è basata su Supabase Postgres. Le entità principali sono `Profile`, `Project`, `Image`, `LaboratoryPipeline` e `LaboratoryPipelineStep`. Questa struttura consente di collegare utenti, progetti, immagini sorgente e cronologie di elaborazione.

4.2.4 AI provider

L'integrazione con i modelli AI è gestita tramite registry e adapter. Il catalogo dei processi espone tre famiglie principali: segmentazione da prompt, rimozione con maschera e generazione da prompt. Gli adapter traducono le richieste interne nel formato richiesto dal provider esterno.

4.3 Pipeline di elaborazione

La pipeline rappresenta la sequenza dei processi applicati a un'immagine. Uno step può generare una maschera, uno step successivo può usare quella maschera per rimuovere un oggetto, e un ulteriore step può generare contenuto tramite prompt. Ogni step mantiene input, output, tipo di processo, modello scelto, impostazioni aggiuntive e stato.

4.4 Scelte tecnologiche

React e Vite sono stati scelti per costruire un'interfaccia reattiva e modulare. FastAPI è stato adottato per la chiarezza nella definizione delle API, la validazione tramite schemi e la facilità di integrazione con servizi Python. Supabase Postgres fornisce un database remoto adatto alla persistenza dei dati applicativi, mentre lo storage immagini permette di mantenere file sorgente e risultati accessibili dal frontend e dai provider AI.

Capitolo 5

Implementazione

L'implementazione segue una separazione netta tra frontend, backend, persistenza e integrazione AI. Questa organizzazione permette di modificare una parte del sistema senza compromettere le altre e rende più semplice aggiungere nuovi processi.

5.1 Struttura del sistema

Il progetto è diviso principalmente in `frontend/` e `backend/`. Il frontend contiene pagine, componenti, hook, tipi TypeScript e client API. Il backend contiene router, servizi, repository, modelli, schemi, processi, registry dei modelli e utility per la gestione delle maschere.

5.2 Frontend

Le pagine di login e signup gestiscono l'accesso dell'utente. La home permette di creare progetti, caricare immagini, visualizzare anteprime e aprire il laboratorio. La gallery offre una visione organizzata degli asset. La pagina laboratory consente di costruire e applicare pipeline di editing. La pagina pipelines permette di consultare elaborazioni salvate, mentre la pagina profile raccoglie le informazioni dell'utente.

5.3 Backend e API

Gli endpoint principali sono organizzati nei router `projects`, `images`, `profile`, `processes` e `laboratory_pipelines`. Le richieste vengono validate tramite schemi, elaborate dai servizi applicativi e persistite tramite repository. Questa separazione riduce l'accoppiamento tra API, logica di business e accesso al database.

5.4 Gestione immagini e progetti

I progetti raggruppano le immagini caricate e le pipeline associate. L'upload salva il file nello storage e registra nel database le informazioni necessarie a recuperarlo. Le immagini possono essere usate come input per il laboratorio e gli output prodotti dai modelli possono essere salvati nello stesso contesto progettuale.

5.5 Pipeline AI

5.5.1 Segmentazione

Il processo `segment_from_prompt` riceve un'immagine e un prompt che descrive l'oggetto o l'area da individuare. Il risultato è una maschera utilizzabile negli step successivi.

5.5.2 Rimozione

Il processo `remove_with_mask` riceve immagine e maschera. Il modello elimina l'area selezionata e ricostruisce il contenuto circostante in modo coerente.

5.5.3 Generazione

Il processo `generate_from_prompt` permette di riempire una regione mascherata seguendo un prompt testuale. Questo consente non solo di cancellare contenuti, ma anche di sostituirli con elementi generati.

5.6 Organizzazione del codice

Il backend separa router API, servizi, repository, processi e registry dei modelli. Il frontend separa pagine, componenti riutilizzabili, hook e client HTTP. Questa struttura rende esplicite le responsabilità: l'interfaccia gestisce l'interazione utente, il backend coordina il workflow, i repository gestiscono la persistenza e gli adapter comunicano con i provider AI.

Capitolo 6

Risultati e Validazione

La validazione considera sia le funzionalità implementate sia la capacità del sistema di rispondere ai requisiti individuati. L'obiettivo è verificare che il prototipo permetta effettivamente di svolgere il workflow previsto: organizzare immagini, applicare processi AI e salvare risultati.

6.1 Funzionalità implementate

Il sistema implementa autenticazione, gestione del profilo, creazione e modifica di progetti, upload di immagini, consultazione della gallery, laboratorio di editing, catalogo dei processi AI, esecuzione di step, anteprima di maschere convex hull e salvataggio delle pipeline. Le funzionalità sono integrate in un flusso utente coerente, dalla selezione dell'immagine fino alla consultazione dei risultati.

6.2 Test

Il backend include test unitari e di integrazione per servizi, dipendenze di autenticazione e API principali. Il frontend include test su client API, hook, pagine e componenti del laboratorio. I test verificano casi come gestione delle risposte HTTP, progetti, profilo, pipeline, selezione di immagini e rendering dei componenti principali.

6.3 Validazione

Rispetto ai bisogni iniziali, la soluzione soddisfa i requisiti di semplicità, automazione e organizzazione. L'utente può eseguire operazioni complesse senza intervenire manualmente sui pixel, mentre il sistema conserva le informazioni necessarie a comprendere la sequenza di elaborazione applicata.

6.4 Risultati

I risultati attesi sono immagini prima/dopo in cui l'oggetto selezionato viene rimosso o sostituito. La qualità dipende da tre fattori principali: precisione della maschera, coerenza del modello generativo e complessità dello sfondo. Su sfondi uniformi il risultato tende a essere più stabile, mentre scene complesse, oggetti parzialmente occlusi o prompt ambigui possono produrre imprecisioni.

Capitolo 7

Conclusioni

Il progetto ha portato alla realizzazione di una piattaforma web capace di integrare gestione delle immagini e workflow AI per l'editing generativo. Magic Eraser dimostra come segmentazione, maschere e inpainting possano essere combinati in un ambiente accessibile e organizzato.

7.1 Valutazione del lavoro

Rispetto al problema iniziale, il sistema riduce la complessità dell'editing e rende disponibili operazioni avanzate attraverso un'interfaccia guidata. La scelta di salvare pipeline e step permette inoltre di analizzare il processo, non soltanto il risultato finale.

7.2 Limiti della soluzione

I principali limiti riguardano la dipendenza da provider AI esterni, la latenza delle richieste, i costi potenziali delle API e la variabilità dei risultati generativi. La qualità finale non è sempre prevedibile e può richiedere più tentativi, soprattutto quando la maschera non delimita correttamente l'area da modificare.

7.3 Contributo

Il contributo del progetto consiste nell'integrazione di funzionalità di editing AI in una piattaforma strutturata per progetti e pipeline. A differenza di strumenti più chiusi, Magic Eraser rende espliciti gli step e crea una base utile per sperimentare modelli, parametri e combinazioni di processi.

7.4 Sviluppi futuri

Gli sviluppi futuri includono un editor manuale delle maschere, confronto tra più modelli, funzionalità di undo e redo, esportazione delle pipeline, metriche quantitative sulla qualità, elaborazione batch, supporto a modelli locali, collaborazione multiutente, versionamento dei risultati e dashboard per monitorare tempi e costi di elaborazione.

Bibliografia

- [1] Kirillov, A. et al. Segment Anything. arXiv:2304.02643, 2023. Disponibile su: <https://segment-anything.com>.
- [2] Rombach, R. et al. High-Resolution Image Synthesis with Latent Diffusion Models. IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022.
- [3] Bertalmio, M., Sapiro, G., Caselles, V. e Ballester, C. Image Inpainting. SIGGRAPH 2000, New Orleans, 2000.
- [4] Suvorov, R. et al. Resolution-robust Large Mask Inpainting with Fourier Convolutions. IEEE/CVF Winter Conference on Applications of Computer Vision, 2022.
- [5] Xu, S. The Best Open-Source Image Generation Models in 2026. BentoML Blog, 2026. Disponibile online sul sito BentoML.
- [6] FastAPI Documentation. Framework Python per la costruzione di API web.
- [7] React Documentation. Libreria JavaScript per la costruzione di interfacce utente.
- [8] Supabase Documentation. Piattaforma backend basata su PostgreSQL, autenticazione e storage.

Appendice A

Manuale Utente

A.1 Login

L'utente accede tramite la pagina di login. In assenza di un account può utilizzare la pagina di signup per creare un nuovo profilo.

A.2 Creare un progetto

Nella home l'utente inserisce il nome del progetto e lo crea. Il progetto diventa il contenitore delle immagini caricate e delle pipeline generate.

A.3 Caricare un'immagine

L'utente seleziona uno o più file, sceglie il progetto di destinazione e avvia l'upload. Le immagini vengono mostrate come anteprime e possono essere aperte nel laboratorio.

A.4 Usare il laboratorio

Nel laboratorio l'utente sceglie il processo da applicare, seleziona il modello, imposta eventuali prompt o parametri e avvia l'elaborazione. Gli output intermedi possono diventare input degli step successivi.

A.5 Salvare una pipeline

Una pipeline può essere salvata con nome, immagine iniziale, immagine finale e lista degli step eseguiti. In questo modo il lavoro rimane consultabile nella sezione pipelines.

Appendice B

API principali

B.1 Elenco endpoint

- `/projects`: creazione, lettura, aggiornamento ed eliminazione dei progetti.
- `/images`: gestione delle immagini caricate.
- `/profile`: lettura e aggiornamento del profilo utente.
- `/processes/catalog`: catalogo dei processi AI disponibili.
- `/processes/run`: esecuzione di un processo AI.
- `/processes/convex-hull-preview`: generazione di una maschera convex hull a partire da una maschera esistente.
- `/laboratory-pipelines`: gestione delle pipeline salvate.
- `/laboratory-pipelines/{id}/steps`: gestione degli step associati a una pipeline.

Appendice C

Configurazione e Avvio

C.1 Docker Compose

Il file `docker-compose.dev.yml` permette di avviare l'ambiente di sviluppo quando si desidera eseguire i servizi in container.

C.2 Variabili ambiente

Il backend richiede le variabili ambiente principali elencate di seguito:

- `DATABASE_URL`
- `SUPABASE_URL`
- `SUPABASE_SERVICE_ROLE_KEY`
- `SUPABASE_STORAGE_BUCKET`

La configurazione verifica che il database punti a Supabase Postgres durante l'esecuzione ordinaria.

C.3 Avvio frontend e backend

Il frontend si avvia dalla cartella `frontend/` con i comandi del progetto Vite. Il backend si avvia dalla cartella `backend/` tramite il server FastAPI/Uvicorn, dopo aver installato le dipendenze e configurato il file ambiente.